CPE 314 — Software Design & Software Design Laboratory

# Week 6 — Event-Driven Programming, Data Mining & Visualization

*Event-Driven and Concurrent Programming | Data Mining | Data Visualization*

## Learning Objectives

- Build a simple GUI that responds to user events using JavaFX.
- Explain the risk of blocking the UI thread and use a background thread correctly.
- Identify and avoid basic concurrency hazards (race conditions, unsynchronized shared state).
- Apply basic data mining operations (filtering, aggregation, frequency analysis) to a dataset.
- Visualize the results of an analysis using a chart, applying good visualization principles.

## 1. Event-Driven and Concurrent Programming

### 1.1 The Event-Driven Model

Instead of executing top-to-bottom, an event-driven program registers handlers and then waits — a central event loop dispatches each user action (click, keypress, timer tick) to the matching handler.

```
Button analyzeBtn = new Button("Analyze");

analyzeBtn.setOnAction(event -> {

    System.out.println("Button clicked!");

    runAnalysis();

});
```

#### 1.1.1 Other Kinds of Events Beyond Clicks

| Event Type | JavaFX Example |
|---|---|
| Mouse | setOnMouseClicked, setOnMouseEntered/Exited |
| Keyboard | setOnKeyPressed, setOnKeyTyped |
| Window | setOnCloseRequest (user clicks the X button) |
| Timer | Timeline + KeyFrame for periodic/animated updates |
| Data change | ListChangeListener, ChangeListener on a Property |

#### 1.1.2 Frequently Confused: Calling a Method vs. Registering a Handler

A very common beginner mistake — especially for students who have seen JavaScript's onClick={func()} anti-pattern, which misbehaves silently at runtime — is writing setOnAction(runAnalysis()) instead of setOnAction(event -> runAnalysis()). In Java, if runAnalysis() is a void method (the normal case), the first

version does not even compile: setOnAction() requires an EventHandler<ActionEvent> object, and the compiler rejects the void result of calling runAnalysis() directly, reporting something like "incompatible types: void cannot be converted to EventHandler<ActionEvent>." The second version hands the button a reference to a block of code to run LATER, whenever the event loop actually detects a click — it compiles because the lambda itself, not runAnalysis()'s return value, is the EventHandler. Registering a handler is not the same as running it: attaching a handler only tells the event loop "call this when the event occurs"; it does not execute anything on the spot. Java's static typing is actually a safety net here — this exact mistake fails LOUDLY at compile time instead of silently doing the wrong thing at runtime.

```
// WRONG: does not compile -- setOnAction() needs an EventHandler<ActionEvent>, not void

analyzeBtn.setOnAction(runAnalysis());

// error: incompatible types: void cannot be converted to EventHandler<ActionEvent>



// RIGHT: hands the button a reference to code to run later, on click

analyzeBtn.setOnAction(event -> runAnalysis());
```

## 1.2 Why You Can't Block the UI Thread

JavaFX (like most GUI toolkits) runs all UI updates on a single thread (the "Application Thread"). If a click handler runs a slow operation directly (e.g., a 5-second computation or a network call), the entire interface freezes until it finishes — no repainting, no responding to further clicks.

### 1.2.1 A Concrete Freeze Example

```
// BAD: freezes the entire application for 5 seconds

analyzeBtn.setOnAction(event -> {

    try { Thread.sleep(5000); } catch (InterruptedException e) {}

    resultLabel.setText("Done!");

});

// During those 5 seconds: no repaint, no other clicks register, window looks "hung"
```

### 1.2.2 Millisecond-by-Millisecond Trace: Blocking vs. Background

It helps to trace exactly what the Application Thread is doing, moment by moment, in both versions of the click handler below.

| Time | Blocking Version (bad) | With a Background Thread (good) |
|---|---|---|
| t = 0 ms | Click detected; handler starts running Thread.sleep(5000) directly | Click detected; handler creates a Task, calls new Thread(task).start(), |

| | | |
|---|---|---|
| | on the UI thread | then returns immediately |
| t = 1-4999 ms | UI thread is stuck inside sleep(); no repaint, no other event can be dispatched | UI thread is free and idle in the event loop; window repaints normally, other buttons respond |
| t = 5000 ms | sleep() finally returns; label text updates; UI thread resumes the event loop | Background thread finishes call(); its result is queued for the UI thread |
| t = 5001 ms | Any clicks made during the freeze are only now being processed, one by one | setOnSucceeded runs on the UI thread and updates the label — the only moment either thread touches the UI |

### 1.2.3 A Second Real-World Analogy: The Single Bank Teller

Picture a bank with exactly one teller window (the Application Thread) and a line of customers (events: clicks, repaints, keypresses). If that teller stops to personally walk to the vault and count out a large loan by hand (a slow task run directly in the handler), every other customer in line — including someone just trying to ask a quick question — is stuck waiting, and the whole branch looks like it isn't functioning. The fix is not to make the teller faster; it's to send the slow loan-processing task to a back-office clerk (a background thread) so the teller window keeps serving the line while the clerk works.

## 1.3 Offloading Work to a Background Thread

```
analyzeBtn.setOnAction(event -> {
    Task<String> task = new Task<>() {
        @Override
        protected String call() {
            return runSlowAnalysis();      // runs on a background thread
        }
    };
    task.setOnSucceeded(e -> resultLabel.setText(task.getValue()));  // back on UI thread
    new Thread(task).start();
});
```

### 1.3.1 Reporting Progress Back to the UI

```
Task<String> task = new Task<>() {
    @Override
```

```
    protected String call() {
        for (int i = 0; i <= 100; i += 10) {
            updateProgress(i, 100);           // safely updates a bound ProgressBar
            try { Thread.sleep(200); } catch (InterruptedException ignored) {}
        }
        return "Analysis complete";
    }
};
progressBar.progressProperty().bind(task.progressProperty());
new Thread(task).start();
```

Task provides updateProgress()/updateMessage() specifically so a background thread can safely push incremental updates to bound UI properties, without manually juggling Platform.runLater() calls yourself.

### 1.3.2 Frequently Confused: Thread vs. Runnable vs. Task

These three names get mixed up constantly. A Runnable is just a bundle of code to run — the WHAT ("here is a block of instructions"), and by itself it does nothing until something executes it. A Thread is an actual OS-backed worker capable of executing code independently — the WHO. A JavaFX Task is a specialized wrapper built on top of this idea specifically for GUI apps: it behaves like a Runnable (you override call()) but also adds safe, built-in channels (updateProgress, updateMessage, setOnSucceeded) for communicating back to the UI thread. Creating a Task object alone does NOT run it in the background — you must still hand it to a Thread and call start(), exactly as shown above.

### 1.3.3 Second Worked Example: Before/After Refactoring a Freezing Search Feature

Consider a "Search Customers" button whose handler queries a (simulated) slow database directly. Before: the whole window locks up while the query runs. After: the same query moves into a Task, and only the final result touches the UI.

```
// BEFORE: freezes the UI for the duration of the query
searchBtn.setOnAction(event -> {
    List<Customer> results = database.slowSearch(nameField.getText());  // 2-3 seconds
    resultsTable.setItems(FXCollections.observableArrayList(results));
});


// AFTER: UI stays responsive while the query runs in the background
```

```
searchBtn.setOnAction(event -> {

    Task<List<Customer>> searchTask = new Task<>() {

        @Override

        protected List<Customer> call() {

            return database.slowSearch(nameField.getText());   // background thread

        }

    };

    searchTask.setOnSucceeded(e ->

        resultsTable.setItems(FXCollections.observableArrayList(searchTask.getValue())));
// UI thread

    new Thread(searchTask).start();

});
```

## 1.4 A Core Concurrency Hazard: Race Conditions

When two threads read and write shared data without coordination, the final result can depend on unpredictable timing. Example: two threads both executing count++ on a shared int can lose an increment, because ++ is really read-increment-write, not one atomic step.

```
// UNSAFE if called from multiple threads:

count++;



// SAFER:

synchronized (lock) {

    count++;

}

// or use AtomicInteger for simple counters
```

### 1.4.1 Why This Matters Even in a "Simple" GUI App

A background Task updating a shared List while the UI thread simultaneously reads that same List to redraw a table is exactly this hazard. The safe pattern is: let the background thread compute a full result, then hand the finished result to the UI thread once (via setOnSucceeded), rather than having both threads touch the same mutable collection concurrently.

### 1.4.2 Deadlock, Briefly

A related hazard: if Thread A holds Lock 1 and waits for Lock 2, while Thread B holds Lock 2 and waits for Lock 1, neither can proceed — a deadlock. Avoid this by always acquiring multiple locks in the same consistent order across your whole codebase.

## 1.5 Quick Self-Check: Event-Driven & Concurrent Programming

1. What is the difference between a program executing top-to-bottom and an event-driven program?
2. Why does putting Thread.sleep(5000) inside a button handler freeze the entire application?
3. Which JavaFX method call actually starts a Task running on a background thread?
4. Why is count++ unsafe when two threads execute it at the same time?
5. What is the one thing setOnSucceeded is allowed to safely do that call() should not do directly?

# 2. Data Mining (Introductory)

## 2.1 What "Data Mining" Means at This Level

At an introductory level, data mining means extracting useful patterns from raw data through filtering, aggregation, and frequency analysis — not full machine learning. The goal is to turn a large, unstructured dataset into a small set of meaningful facts.

## 2.2 Example: Word Frequency Mining

```
Map<String, Long> frequency = Arrays.stream(text.toLowerCase().split("\\W+"))
        .filter(w -> !w.isBlank())
        .collect(Collectors.groupingBy(w -> w, Collectors.counting()));


frequency.entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .limit(5)
        .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
```

### 2.2.1 Trace: Building the Frequency Tally Step by Step

It's worth tracing the underlying algorithm by hand once, without streams, to see exactly how the Map<String, Integer> tally builds up. Suppose the input list is ["cat", "dog", "cat", "bird", "dog", "cat"]:

| Step | Word Read | Map State After This Step |
|---|---|---|
| 1 | cat | {cat: 1} |
| 2 | dog | {cat: 1, dog: 1} |
| 3 | cat | {cat: 2, dog: 1} |
| 4 | bird | {cat: 2, dog: 1, bird: 1} |

| | | |
|---|---|---|
| 5 | dog | {cat: 2, dog: 2, bird: 1} |
| 6 | cat | {cat: 3, dog: 2, bird: 1} |

The rule at every step is the same: if the word is already a key in the map, increment its value by 1; otherwise, insert it with a starting value of 1. That single rule, applied once per word, is the entire frequency-counting algorithm — Collectors.groupingBy(...,Collectors.counting()) is simply Java's built-in shorthand for it.

```
Map<String, Integer> tally = new HashMap<>();

for (String word : words) {

    tally.merge(word, 1, Integer::sum);   // +1 if present, insert with 1 if new

}
```

## 2.3 Example: Aggregating Sales Data

```
Map<String, Double> totalByCategory = sales.stream()

        .collect(Collectors.groupingBy(Sale::getCategory,

                Collectors.summingDouble(Sale::getAmount)));
```

## 2.4 Example: Finding a Trend Over Time

```
Map<Month, Double> spendingByMonth = transactions.stream()

        .collect(Collectors.groupingBy(t -> t.getDate().getMonth(),

                Collectors.summingDouble(Transaction::getAmount)));

// Sorting by Month (a natural enum order) reveals the trend across the year
```

## 2.5 Filtering Before Mining: Garbage In, Garbage Out

Real datasets contain noise: empty strings, malformed rows, outlier values from data-entry errors. Always filter obviously invalid data before aggregating — an aggregation over dirty data produces a confident-looking but meaningless number.

```
List<Transaction> clean = raw.stream()

        .filter(t -> t.getAmount() > 0)          // discard invalid/negative entries
```

```
        .filter(t -> t.getDate() != null)        // discard rows with missing dates
        .collect(Collectors.toList());
```

### 2.6 Frequently Confused: Aggregation vs. Filtering

Students often use these two words interchangeably, but they do different jobs and are usually used together in sequence. Filtering REDUCES the row count by keeping only rows that match a condition — it never changes the shape of the remaining data, each surviving row still looks like a normal row. Aggregation REDUCES the row count by COMBINING many rows into a smaller number of summary values (a sum, a count, an average per group) — the output no longer looks like the original rows at all. In practice you almost always filter first (remove bad or irrelevant rows) and aggregate second (summarize what remains), as shown in the clean transactions example above.

- Filtering: 1,000 transactions -> 950 transactions (still individual transaction rows)
- Aggregation: 950 transactions -> 12 monthly totals (rows collapsed into summaries)

### 2.7 Quick Self-Check: Data Mining

6. In one sentence, what is the difference between filtering and aggregating?
7. In the word-tally trace, what happens to the map when a word is seen for the first time versus the second time?
8. Why should invalid rows be filtered out before an aggregation, not after?
9. What Java Collectors call produces a Map<String, Long> of counts grouped by key?

## 3. Data Visualization

### 3.1 Principles of Good Visualization

- Choose the right chart type for the data (trend over time → line chart; comparison across categories → bar chart; part-to-whole → pie chart, used sparingly).
- Label axes and provide a legend — a chart nobody can interpret has no value.
- Avoid chartjunk — unnecessary 3D effects, excessive colors, or decoration that doesn't convey information.
- Keep scales honest — a truncated y-axis can make a small difference look enormous.

#### 3.1.1 Frequently Confused: Bar Chart vs. Histogram

These look similar (both use vertical bars) but answer different questions, and mixing them up is one of the most common visualization mistakes. A BAR CHART compares separate, independent, usually categorical values (e.g., total sales per product category) — the categories have no inherent numeric order, so gaps between bars are normal and the bar order can be rearranged freely. A HISTOGRAM shows the distribution of a single continuous numeric variable by grouping values into ranges called bins (e.g., how many students scored 70-79, 80-89, 90-100) — the bins DO have a numeric order, bars are drawn touching with no gaps to show that the underlying scale is continuous, and reordering the bars would misrepresent the data. If your x-axis labels are category names, you almost certainly want a bar chart, not a histogram.

## 3.2 Building a Chart in JavaFX

```
CategoryAxis xAxis = new CategoryAxis();

NumberAxis yAxis = new NumberAxis();

BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);

chart.setTitle("Top 5 Words by Frequency");



XYChart.Series<String, Number> series = new XYChart.Series<>();

frequency.forEach((word, count) -> series.getData().add(new XYChart.Data<>(word, count)));

chart.getData().add(series);
```

### 3.2.1 Building a Line Chart for Trend Data

```
NumberAxis xAxis = new NumberAxis();

NumberAxis yAxis = new NumberAxis();

LineChart<Number, Number> trendChart = new LineChart<>(xAxis, yAxis);

trendChart.setTitle("Monthly Spending Trend");



XYChart.Series<Number, Number> series = new XYChart.Series<>();

series.setName("2026 Spending");

spendingByMonth.forEach((month, total) ->

        series.getData().add(new XYChart.Data<>(month.getValue(), total)));

trendChart.getData().add(series);
```

### 3.2.2 A Second Real-World Analogy: Choosing a Chart Is Like Choosing a Camera Lens

A photographer doesn't use the same lens for every shot. A wide-angle lens captures the whole landscape and how it changes across the scene — this is a line chart, built for showing a trend across a continuous range like time. A macro lens isolates and compares fine detail on a few distinct subjects side by side — this is a bar chart, built for comparing separate categories. A fisheye lens distorts the scene for dramatic effect — this is what a truncated axis or an unnecessary 3D pie chart does to your data: it may look striking, but it misrepresents the underlying reality. Just as a photographer picks the lens that serves the shot, a developer should pick the chart type that serves the data, not the one that looks most impressive.

### 3.3 Connecting the Three Topics

This week's pipeline mirrors a realistic mini-application: an event-driven GUI triggers a background thread that mines a dataset for patterns, and the result is rendered back on the UI thread as a chart — the same pattern used in Week 8-11's project.

```
[Button Click]  --event-->  [Background Task: filter + aggregate + mine]
                                          |
                                 setOnSucceeded (back on UI thread)
                                          |
                                          v
                                [Chart updated with mined result]
```

### 3.4 Quick Self-Check: Data Visualization

10. When would you choose a line chart over a bar chart?
11. What is the key structural difference between a bar chart and a histogram?
12. Why is a y-axis that starts at 950 instead of 0 considered misleading rather than just a style choice?
13. Give one example of chartjunk and explain why it hurts rather than helps.

## 4. Assignment 5

This assignment is completed INDIVIDUALLY — there is no pair/partner work this term. Each student designs, writes, and submits their own solution demonstrating this week's four ideas: an event-driven GUI, a background thread that keeps the UI responsive, a data-mining aggregation step, and a chart that visualizes the result.

### Task 1: Responsive Score Statistics Viewer (JavaFX or Swing)

Build a small GUI with a text field or text area where the user enters a list of numeric quiz scores (comma- or newline-separated) and a "Compute Statistics" button. Clicking the button must run the statistics computation (minimum, maximum, average, and the count of scores in each of three ranges: 0-59, 60-79, 80-100) on a background thread/Task, simulating a slow computation with a short artificial delay. While the computation runs, a separate on-screen clock or tick counter (updated on a short JavaFX Timeline or Swing Timer) must keep visibly updating, proving the UI thread was never blocked. The final statistics are written to a results Label only once the background work finishes.

### Task 2: Category Frequency Dashboard

Using a small provided (or self-authored) list of category labels (e.g., customer feedback tags such as "Shipping", "Quality", "Price", including a few blank/invalid entries mixed in), build a second feature — in the same app or a separate small app — that: filters out blank/invalid entries first, tallies how often each

remaining category appears using a frequency map, performs this tally inside a background Task, and then displays the resulting frequencies as a BarChart once the task completes successfully.

## Acceptance Criteria

- ☐ Task 1's statistics computation happens entirely inside Task.call() (background thread), not directly inside the button's event handler
- ☐ Task 1's on-screen clock/tick counter keeps updating while the statistics computation is running, proving the UI thread stayed free
- ☐ Task 2 filters blank/invalid category entries BEFORE tallying frequency, not after
- ☐ Task 2's frequency tally happens inside Task.call(); the BarChart is only built or updated inside setOnSucceeded (UI thread)
- ☐ Code compiles and runs without freezing the window at any point during either task
- ☐ You can explain, in your own words, which line(s) run on the UI thread and which run on the background thread

Deliverable: Java source file(s), submitted individually through the class repository under your own name — no pair submissions — with a 2-3 sentence note explaining how you kept the UI responsive in both tasks.

## How to Submit (Git)

This assignment is submitted as a branch + pull request in your own cpe314-<edp>-coursework-<your-username> repo — the same repo you reuse all term. Follow the five steps below exactly; the full reference and FAQ live in 00_GitHub_Workflow/Weekly_Assignment_Submission_Guide.md.

### Step 1 — Create Your Branch

Make sure main is up to date, then branch off it:

```
git checkout main

git pull

git checkout -b assignment/week06
```

### Step 2 — Set Up This Week's Folders

Create both task folders up front, inside this week's folder:

```
mkdir -p assignments/week06/task1

mkdir -p assignments/week06/task2
```

task1/ holds Task 1 (Responsive Score Statistics Viewer); task2/ holds Task 2 (Category Frequency Dashboard). Add a short assignments/week06/README.md once you're ready to submit — 2–3 lines on what's finished and what (if anything) is incomplete or shaky.

### Step 3 — Commit Your Work

Write, compile, and run each task locally before committing it. Commit each task separately:

```
git add assignments/week06/task1/

git commit -m "feat(week06-task1): add responsive statistics viewer with background Task"



git add assignments/week06/task2/

git commit -m "feat(week06-task2): add category frequency dashboard with BarChart"
```

Commit at least once per task — more if a task itself produces several files or stages (e.g., one commit per class). A task that's genuinely one small class can be one honest commit; don't squash a multi-class task into a single end-of-week commit.

### Step 4 — Push and Open the Pull Request

```
git push -u origin assignment/week06
```

On GitHub: Compare & pull request → base main → title exactly "Week 06 Assignment — <Last Name, First Name>". The PR template asks what this covers and what (if anything) is incomplete — answer honestly for both tasks.

### Step 5 — Do Not Merge Your Own PR

Leave the PR open once it's submitted — it gets merged into main once graded, which is what keeps every week's assignments/ folder accumulating in the same repo over the term. Pushing more commits to an already-open PR after the deadline is fine; opening the PR is what counts as your submission time.

### PR Checklist

- ☐ assignments/week06/task1/ and task2/ both exist and both compile/run
- ☐ At least one commit per task, with (week06-taskN) in the message
- ☐ assignments/week06/README.md says what's done and what (if anything) isn't
- ☐ PR title is exactly "Week 06 Assignment — <Last Name, First Name>"
- ☐ PR opened before the deadline (further pushes after are fine)
- ☐ You did not merge it yourself